SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

삼성청년
SW·AI아카데미

backend - EL/JSTL

SAMSUNG SW AI ACADEMY FOR YOUTH

목차

1. MVC 리뷰
2. EL
3. JSTL
4. include directive

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

MVC 리뷰

MVC 디자인 패턴

Confidential

❖ Model 2 Deep Dive

근처 맛집!



Controller: Servlet

주요 업무

- HttpServletRequest 분석
- 파라미터를 이용해 적절한 Service 호출
- Model의 반환 값 확인
- 결과를 적절한 웹 스코프에 attribute로 저장
- 상황에 따라 적절한 View 호출

View: JSP

주요 업무

- 웹 스코프에 저장된 attribute를 활용해 화면 생성
- 클라이언트에 화면 전달

Model: Java

Service

주요 업무

- 비즈니스 로직 처리
- 필요시 DAO 호출로 DB 연동

DAO

주요 업무

- JDBC를 이용해 DB 연동

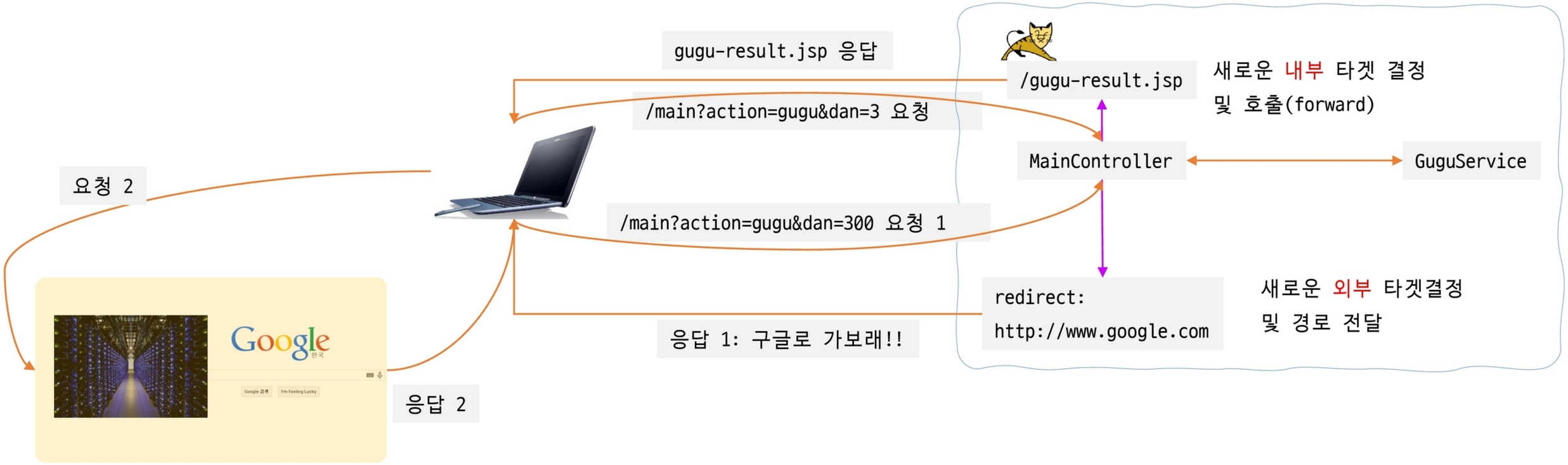
jdbc

DB

웹 컴포넌트 호출

Confidential

❖ Forward vs redirect



웹 컴포넌트 호출

Confidential

❖ Forward vs redirect

구분	Forward	Redirect
클라이언트 URL	변하지 않음	변함
서버 내부 이동	같은 request 범위 : 빠름!	새로운 HTTP 요청 : 느림 (왕복 통신)
request 객체	같은 객체 공유 - Attribute, Parameter 모두 유지	새로운 객체 - Attribute 유지 X, Parameter만 URL 쿼리스트링 형태로 전달 가능
사용 시기	데이터 처리 후 결과 화면 표시	작업 완료 후 다른 페이지로 이동
	단순히 JSP를 호출하는 경우	로그인 -> 인증 완료 -> 메인 페이지로 redirect
	검색 -> DB 조회 -> 조회 결과 확인	삭제 요청 -> 삭제 처리 -> 목록으로 redirect
		글 등록 요청 -> 등록 처리 -> 목록으로 redirect
	에러 발생 -> 기존 정보 활용 -> forward	에러 발생 -> 초기 상태 희망 -> redirect

회원가입(TODO: 01)

Confidential

❖ 회원 가입을 위한 DAO

```
public class FileMemberDao implements MemberDao {
  // singleton 처리
  private final File dataFile;
  private List<Member> members;

  private FileMemberDao() {
    // 파일 초기화
  }

  @Override
  public int insert(Connection con, Member member) {
    synchronized (members) {
      if (members.stream().anyMatch(m -> m.getEmail().equals(member.getEmail()))) {
        throw new RuntimeException("이미 가입된 회원입니다: " + member.getEmail());
      }
      member.setMno(System.currentTimeMillis()); // 회원 번호 자동 부여
      members.add(member);
      return 1;
    }
  }

  // 파일 처리
}
```

multi thread에서 호출된다!



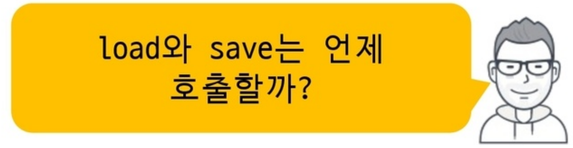
회원가입(TODO: 02)

Confidential

❖ 회원 가입을 위한 DAO

```
public class FileMemberDao implements MemberDao {
  . . .
  private void load() {
    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(dataFile))) {
      members = (List<Member>) ois.readObject();
      System.out.println("회원 정보 로딩 완료: " + members.size() + "명");
    } catch (Exception e) {
      System.out.println("저장된 회원 정보가 없습니다.");
      members = Collections.synchronizedList(new ArrayList<>());
    }
  }

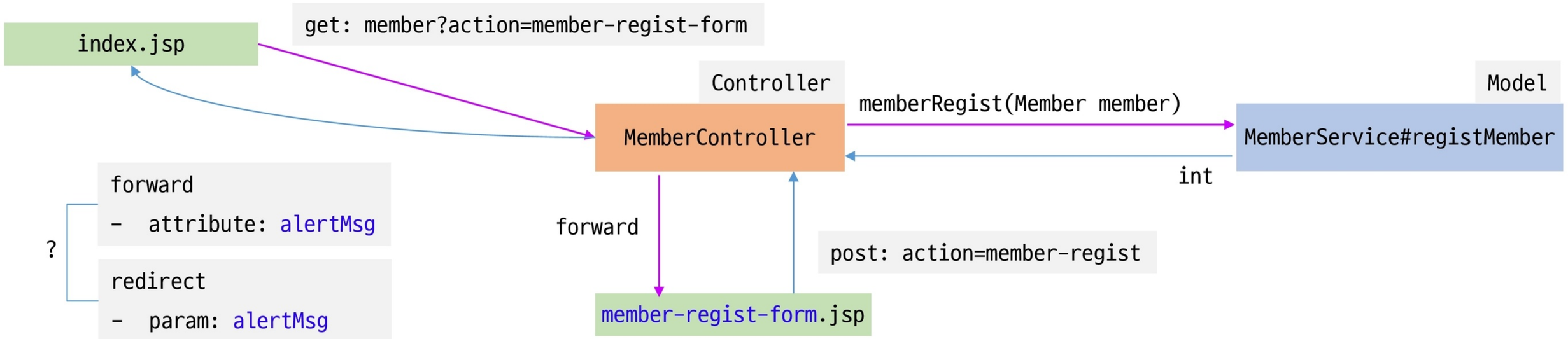
  public void save() {
    try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(dataFile))) {
      oos.writeObject(members);
      System.out.println("회원 정보 저장 완료");
    } catch (Exception e) {
      throw new RuntimeException("회원 정보 저장 실패", e);
    }
  }
}
```



회원가입(TODO:03 ~ 05)

Confidential

❖ 회원 가입: 정상 처리



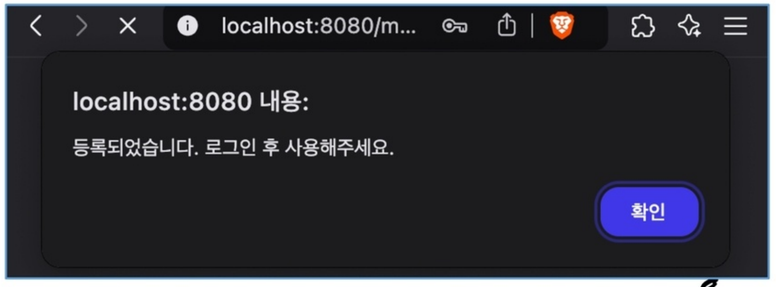
index.jsp



member-regist-form.jsp



index.jsp

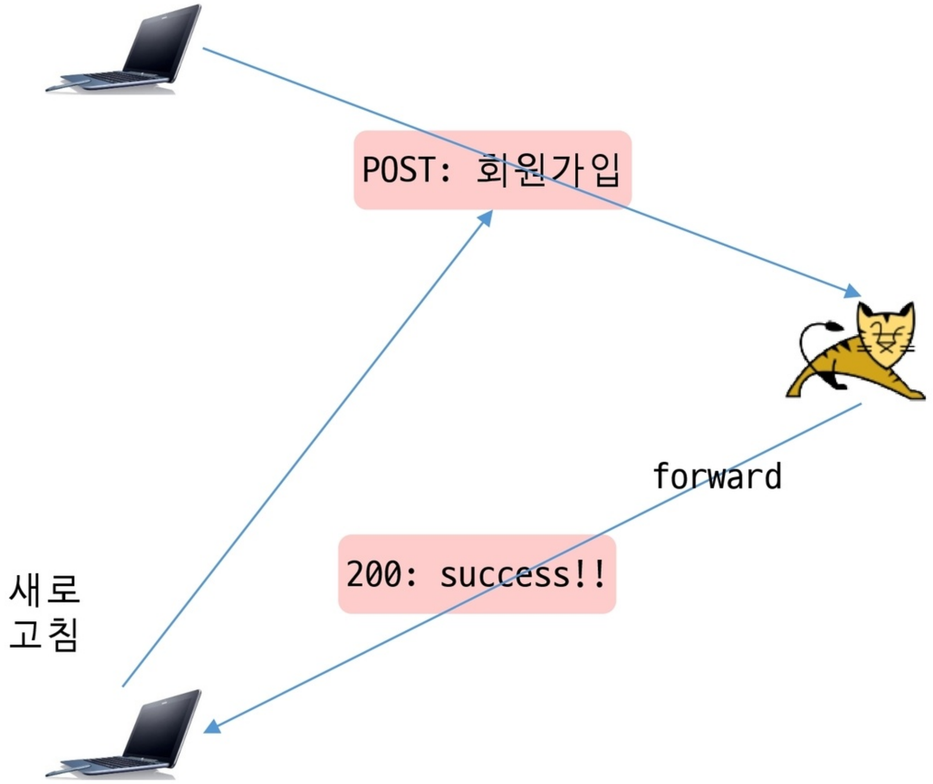


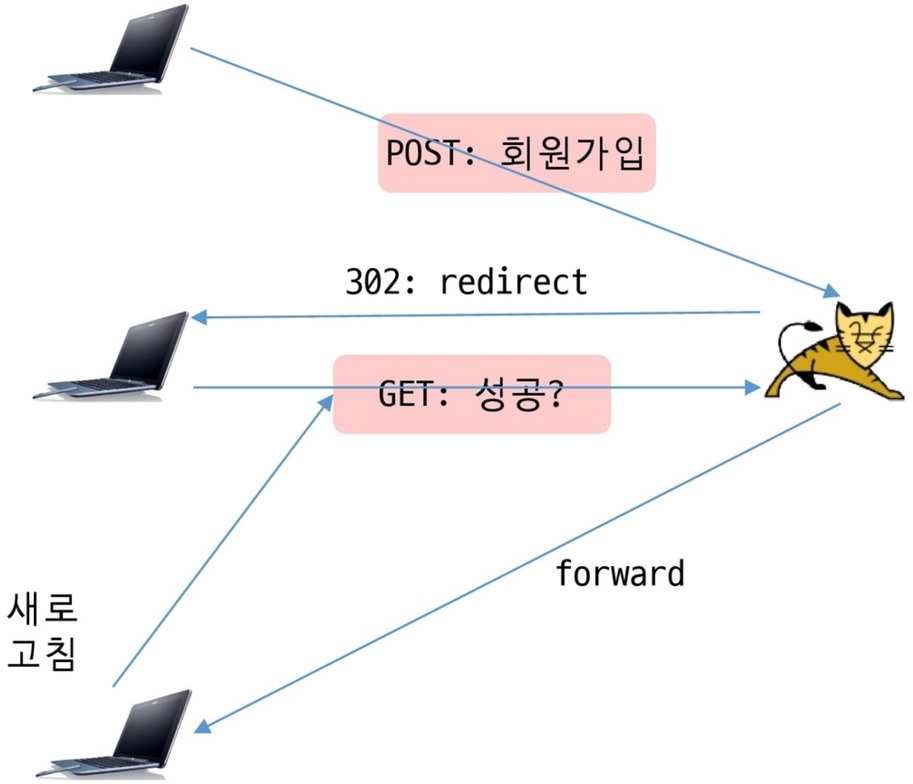
회원가입

Confidential

❖ PRG(Post Redirect Get pattern)

- Post로 받은 요청은 redirect를 이용해서 Get으로 돌려라!
- [새로고침] 행위는 이전에 했던 요청을 다시 진행
  - ◆ 멱등한 GET, PUT, DELETE는 문제 없지만 Post 요청은 멱등하지 않으므로 여러 번 수행 되면 중복 제출 문제 발생

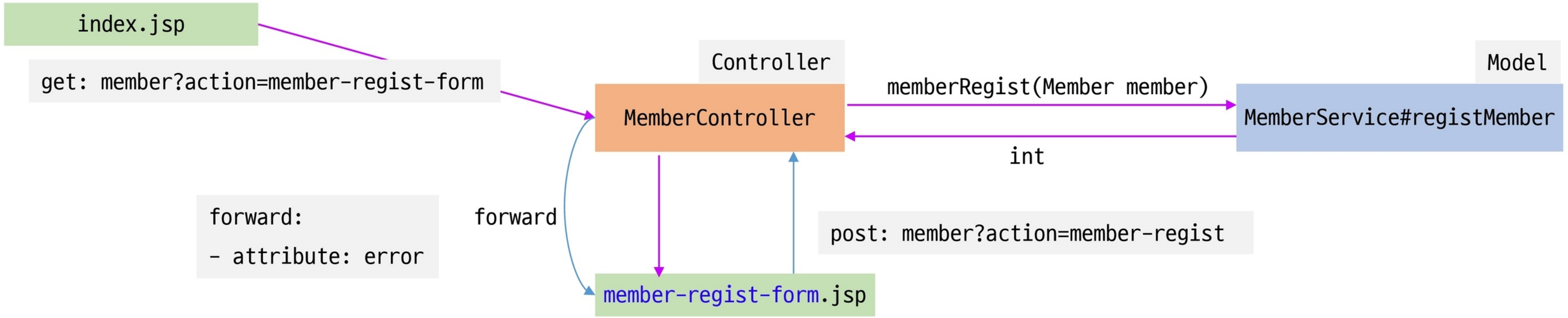




회원가입(TODO: 06, 07)

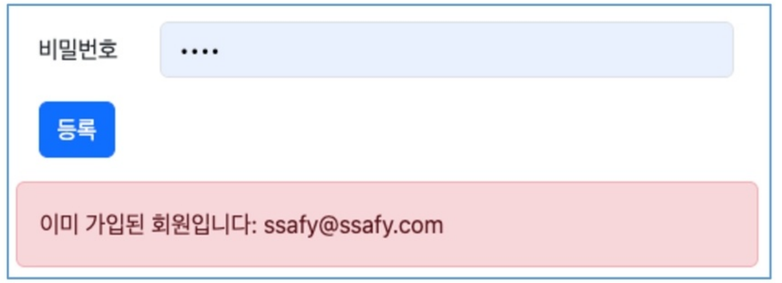
Confidential

❖ 회원 가입: 오류 상황









SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

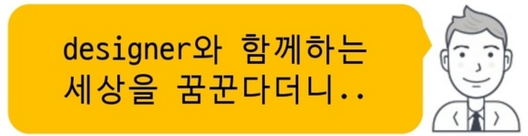
EL

Confidential

❖ EL & JSTL

- JSP를 좀 더 JSP답게 만드는 요소

```
<%
Object error = request.getAttribute("error");
  if(error !=null){
    out.println("<div class='alert alert-danger' role='alert'>"+error+"</div>");
  }
%>
```



- JSP에서 최대한 프로그래밍 요소 제거
  - ◆ 많은 부분을 "진짜로" 태그 중심으로 변경 - designer 또는 publisher 등이 쉽게 접근 가능
  - ◆ EL(Expression Language)
    - 표현 즉 출력을 위한 언어로 JSP의 expression(<%=. . .%>) 대체
    - 단순한 출력은 물론 특히 웹 스코프에 저장된 attribute를 사용하는데 매우 편리
  - ◆ JSTL : Jsp Standard Tag Library
    - 자주 사용되는 기능들에 대해 정형화된 태그 제공
    - 별도의 라이브러리 추가 설치 필요

Confidential

❖ Before & After

```
<%
  request.setAttribute("dan", Integer.parseInt(request.getParameter("dan")));
%>
<%
  Object danObj = request.getAttribute("dan");
  if(danObj!=null){
    int dan = (Integer)danObj;
%>
<table>
  <tbody>
    <%
      for(int i=1; i<10; i++){
          out.append("""
          <tr>
            <td>%d * %d</td><td>%d</td>
          </tr>""".formatted(dan, i, dan*i));
      }// for
    %>
  </tbody>
</table>
<%
}// if
%>
```

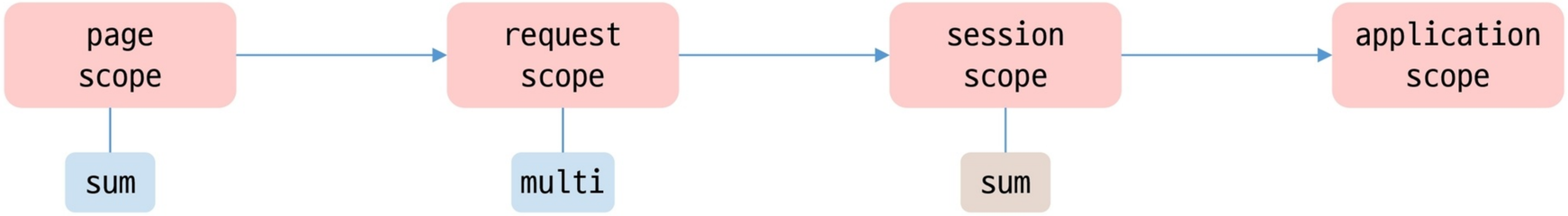
```
<c:set var="dan" value="${param.dan}" />
<c:if test="${!empty dan }">
  <table>
    <tbody>
      <c:forEach var="i" begin="1" end="9" >
        <tr>
          <td>${dan}* ${i }</td>
          <td>${dan*i }</td>
        </tr>
      </c:forEach>
    </tbody>
  </table>
</c:if>
```

EL(TODO: 08,09)

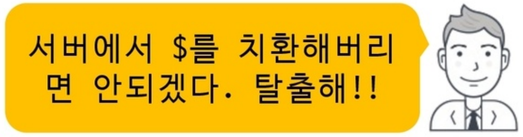
Confidential

❖ EL 기본

- 4가지 웹 스코프에 저장된 attribute를 쉽게 출력하기 위함
- 기본 표현법: ${attribute_name}
- 작은 범위에서 큰 범위로 확장하며 attribute의 검색 → 처음 발견된 값 활용



  - ◆ ${sum} 은 page scope의 값!
  - ◆ session scope의 값을 사용하려는 경우 명시적으로 scope 지정: ${sessionScope.sum}
- 값이 없을 경우 null이 아닌 공백으로 표시



- javascript 템플릿 문자열에서 사용된 $와 혼용 주의
  - ◆ el의 $: 서버 측 표현식으로 서버에서 attribute로 변경 후 client 로 전송함
  - ◆ javascript의 $: 서버와 무관하며 온전히 클라이언트에 있어야 의미가 있음 - escape 처리 필요

Confidential

❖ EL 내장 객체

- JSP에서처럼 EL이 기본적으로 가지고 있는 객체들

이름	객체 타입	표현 데이터	사용 예
pageContext	PageContext	현재 페이지의 컨텍스트	${pageContext}
pageScope	Map	현재 페이지 스코프의 속성	${pageScope.attributeName}
requestScope	Map	현재 요청 스코프의 속성	${requestScope.attributeName}
sessionScope	Map	현재 세션 스코프의 속성	${sessionScope.attributeName}
applicationScope	Map	애플리케이션 스코프의 속성	${applicationScope.attributeName}
param	Map	요청 파라미터의 단일 값	${param.parameterName}
paramValues	Map	요청 파라미터의 여러 값	${paramValues.parameterName}
cookie	Map	현재 요청의 쿠키 정보	${cookie.cookieName}
header	Map	요청 헤더의 단일 값	${header.headerName}
headerValues	Map	요청 헤더의 여러 값	${headerValues.headerName}

Confidential

❖ 객체 접근법 (TODO: 10)

- 객체의 property를 통해 접근
  - ◆ 직접 필드에 접근하는 것이 아니라 getter/setter를 통해서 접근하는 방식
  - ◆ Record는 그냥 field 이름 사용
- Map 계열은 key로 접근

framework 등에서도 property로 접근하기 때문에 private field에 대한 setter/getter는 선택이 아닌 필수!



- JavaScript와 마찬가지로 '.' 표기법 또는 '[ ]' 표기법 사용 가능

표기법	설명	활용 예
. : ${obj.property}	객체지향적이며 간단 - java-naming 룰에 어긋나면 사용 불가	${paramValues.names[0]} ${header.User-Agent} (X)
[ ] : ${obj[property]}	property의 형태에 상관 없이 사용 가능 - 약간 귀찮음	${paramValues["names"][0]} ${header["User-Agent"]}

Confidential

❖ 객체의 메서드 호출

- 객체의 property 뿐 아니라 일반 메서드도 사용 가능

```
List<String> friends=List.of("hong", "jang", "lim");
request.setAttribute(“friends",friends);

<li>friends 수: ${friends.size() }, hong 포함 여부? ${friends.contains("hong") }</li>
```

Confidential

❖ EL의 연산자

구분	연산자	활용 예
산술 연산	+, -, *, /, div, %, mod	
비교 연산	<,lt, >, gt, <=, le, >=, ge, ==, eq, !=, ne	${status ==false }
논리 연산	&&, ¦¦, !, and, or, not	
삼항 연산	? :	${empty param.ID ? "guest" : param.ID }
empty 연산	empty : 데이터의 존재 여부를 나타내는 단항 연산자	${empty name}, ${!empty name}

- 자바와 다른 점
  - ◆ 문자열에 대한 +(결합) 연산은 지원하지 않고 자동 캐스팅 : ${“1”+”2”} : 3, ${“1”+”b”} : 오류
  - ◆ 나누기(/, div) : 정수 간의 연산에도 소수점의 결과를 리턴
  - ◆ 비교 연산자 : 수치 비교뿐 아니라 문자 데이터에 대한 사전식 비교 처리
    - ${"CAT" > "DOG"} → false
    - ${gender =="female"} → attribute의 값과 literal 비교
- empty 값이 true로 평가받는 경우
  - ◆ null, 빈 문자열, 길이가 0인 배열, 빈 Collection 객체

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

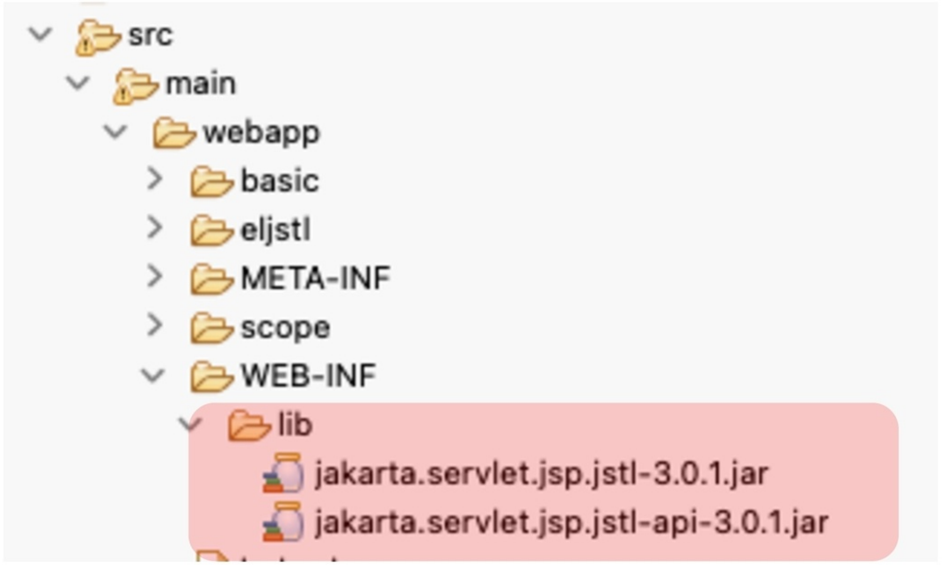
JSTL

Confidential

❖ JSTL

- Tag Library
  - ◆ 개발자가 특정 비즈니스 로직이나 기능 수행을 위한 태그를 직접 정의해서 사용하는 것 - 사용자 정의 태그
- JSP Standard Tag Library
  - ◆ Tag Library 중 자주 사용되는 기능을 모아서 제공하는 태그 모음
  - ◆ 주요 기능: 코어(변수 선언, 조건문, 반복문), 데이터 포맷팅, XML 처리, 데이터베이스 활용, 국제화 등
  - ◆ 주로 EL과 함께 사용되며 Script-let 대비 간결한 페이지 작성에 유리

- 라이브러리 설치
  - ◆ maven
    - 인터페이스: jakarta.servlet.jsp.jstl/jakarta.servlet.jsp.jstl-api
    - 구현체: org.glassfish.web/jakarta.servlet.jsp.jstl
  - ◆ 직접 설치: src/main/webapp/WEB-INF/lib에 저장



Confidential

❖ 라이브러리 추가

- https://mvnrepository.com/



1. **Jakarta Standard Tag Library API** **586** usages

jakarta.servlet.jsp.jstl » jakarta.servlet.jsp.jstl-api GPL EPL

Jakarta Standard Tag Library API

Last Release on Aug 22, 2024



2. **Jakarta Standard Tag Library Implementation**

org.glassfish.web » jakarta.servlet.jsp.jstl EPL GPL **75** usages

Jakarta Standard Tag Library Implementation

Last Release on Nov 20, 2022

```
<dependency>
  <groupId>jakarta.servlet.jsp.jstl</groupId>
  <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
  <version>${jakarta.servlet.jsp.jstl-api.version}</version>
</dependency>

<dependency>
  <groupId>org.glassfish.web</groupId>
  <artifactId>jakarta.servlet.jsp.jstl</artifactId>
  <version>${jakarta.servlet.jsp.jstl.version}</version>
</dependency>
```

Confidential

❖ JSTL 사용 선언

- taglib directive 활용

```
<%@ taglib prefix="c" uri="jakarta.tags.core"%>
```

  - ◆ prefix: 페이지 내에서 해당 태그를 사용할 때 붙이는 접두사
  - ◆ uri: 해당 태그 라이브러리를 식별하는 고유 문자열로 태그라이브러리 로드에 사용

라이브러리	주요 기능	일반적인 접두어	URI
core	변수 지원, 제어문, URL 처리	c	jakarta.tags.core
formatting	지역, 메시지 형식, 숫자 및 날짜 형식	fmt	jakarta.tags.fmt
function	connection 처리, String 처리	fn	jakarta.tags.functions
xml	xml 코어, 흐름 제어, XML 변환	x	jakarta.tags.xml
database	SQL 처리	sql	jakarta.tags.sql

  - ◆ MVC model2 방식에서 JSP는 단순히 view의 역할만 수행 - 거의 core와 formatting 만 사용

Confidential

❖ 변수 지원 : <c:set>, <c:remove>

- 웹 scope에 el 변수(attribute) 관리
- 작성 방법

```
<c:set var="변수명" value="값" [scope="영역"]/>        <c:set var="변수명" [scope="영역"]>값</c:set>
```

  - ◆ var : 값을 지정할 el 변수의 이름으로 타입은 String
  - ◆ value : 변수의 값 자체 → 표현식, EL, 정적인 텍스트를 이용하여 값을 지정하며 타입은 Object
  - ◆ scope : 변수를 저장할 영역(page, request, session, application)으로 기본 값은 page

```
<%request.setAttribute("root", request.getContextPath());%>
<a href="<%=request.getAttribute("root")%>/member?action=member-regist-form">회원가입</a> <br>


<c:set var="root" value="${pageContext.request.contextPath }"/>
<a href="${root}/member?action=member-regist-form">회원가입</a> <br>
```

Confidential

❖ 변수 지원 : <c:set>, <c:remove> (TODO: 11)

- 웹 scope에 el 변수(attribute) 관리
- 작성 방법

```
<c:remove var="변수명" [scope="영역"]/>
```

◆ var : 값을 지정할 el 변수의 이름으로 타입은 String

◆ scope : 변수를 저장할 영역(page, request, session, application)으로 기본 값은 page

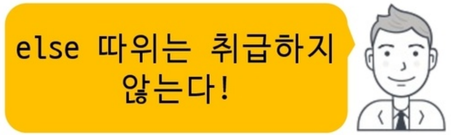
```
<%
  request.setAttribute("name", "hong");
  out.println("name 속성 출력 : " + request.getAttribute("name") + "<br>");
  request.removeAttribute("name");
  out.println("name 속성 출력 : " + request.getAttribute("name") + "<br>");
%>

<c:set var="name" value= "hong" scope="request" />
저장한 \${name } 출력 : ${name }
<br>
<c:remove var="name" scope="request" />
삭제한 \${name } 출력 : ${name }
```

Confidential

❖ 조건문: <c:if> (TODO: 12)

```
<c:if test="조건" [var="test_result"]>
  . . .
</c:if>
```

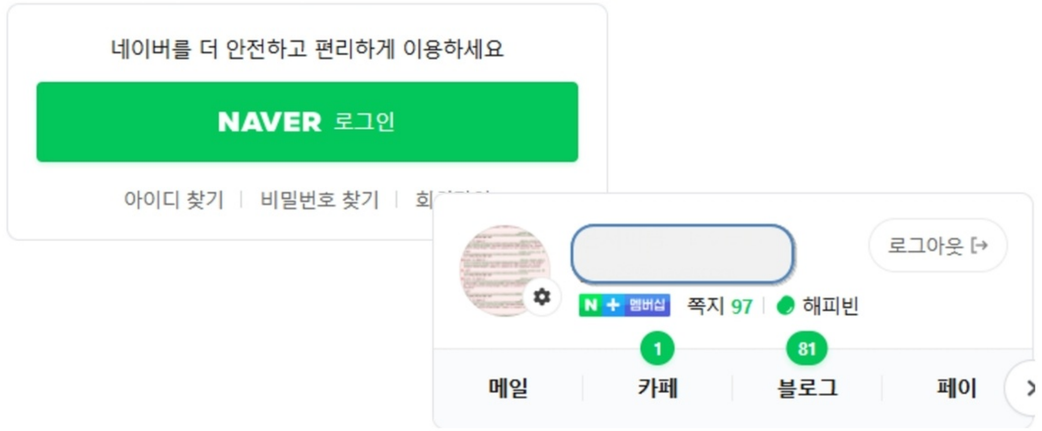


- test attribute에는 true/false 값이나 이를 리턴하는 표현식, EL 사용 가능
  - ◆ 그 외의 값이 오면 false로 인식
- var="test_result"의 경우 test의 결과 값을 test_result을 EL 변수에 저장 해서 재사용 가능

- 상황에 따라 화면을 달리 표현해야 하는 경우 el empty 연산자와 함께 자주 사용됨

```
<c:set var="name" value="hong"/>

<c:if test="${!empty name }">
  ${name }님 반갑습니다.
</c:if>
<c:if test="${empty name }">
  이름을 입력해주세요.
</c:if>
```



Confidential

❖ 조건문: <c:choose>, <c:when>, <c:otherwise> (TODO: 13)

```
<c:choose>
  <c:when test="…"> … </c:when>
  <c:when test="…"> … </c:when>
  <c:otherwise> … </c:otherwise>
</c:choose>
```

- if-else 블록의 형태로 다수의 조건문을 하나의 블록에서 수행할 때 사용

```
<%
  int age = 17;
%>
```




```
<c:choose>
  <c:when test="<%=(age > 18)%>">성인</c:when>
  <c:when test="<%=(age > 10)%>">미성인</c:when>
  <c:otherwise>유아</c:otherwise>
</c:choose>
```

Confidential

❖ 반복문: <c:forEach> (TODO: 14)

- 주어진 횟수 동안 반복 또는 배열, 컬렉션의 내용을 순차 조회

```
<c:forEach var="변수명" items="대상컬렉션" varStatus="status"> …</c:forEach>
<c:forEach var="변수명" begin="1" end="10" [step="2"]> … </c:forEach>
<c:forEach var="변수명" items="대상컬렉션" begin="2" end="10" step="2"> … </c:forEach>
```

- ◆ items: 대상 collection
- ◆ var : 대상 collection에서 해당 반복에 선택된 값 저장
- ◆ varStatus: 루프 정보를 담은 LoopTagStatus 타입의 객체로 index, count 등 정보 포함

```
<%
  Map<String, String> map = Map.of("name", "andy","age", "20");
  request.setAttribute("person", map);
%>

<c:forEach var="data" items="${person }" varStatus="status">
  ${status.index}번째 요소 : ${data.key}, ${data.value}<br>
</c:forEach>
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

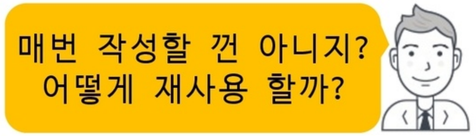
include directive

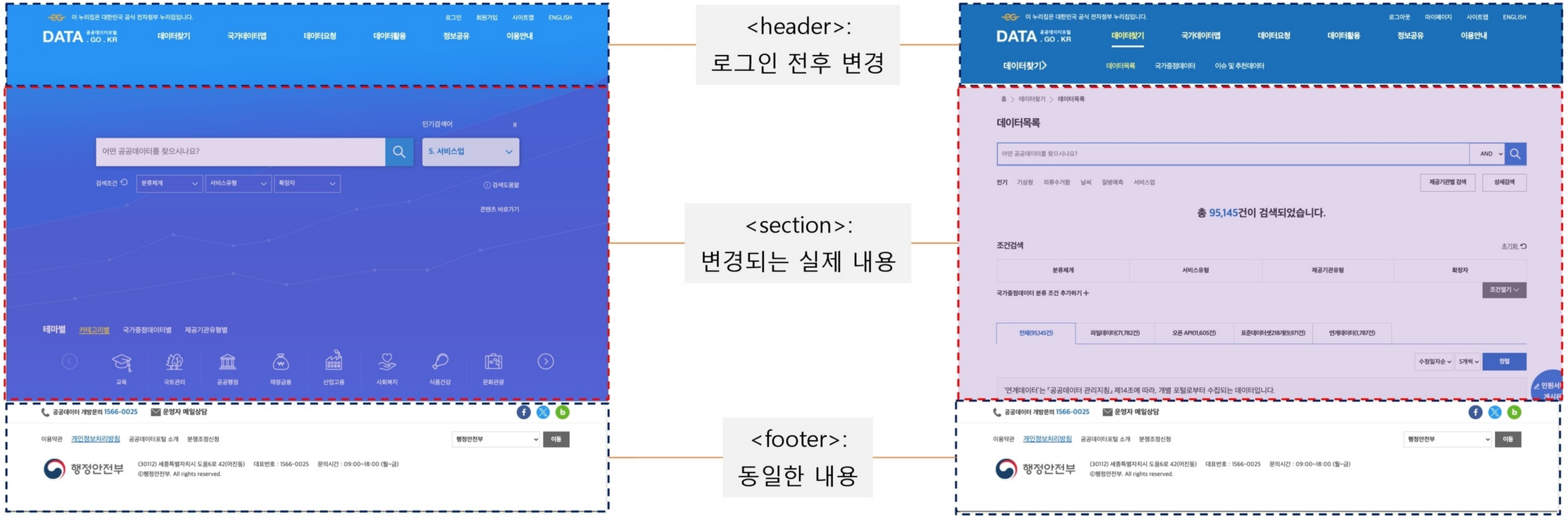
페이지 모듈화

Confidential

❖ 사이트들은 대부분 반복적인 구조를 가지고 있음

- header, footer 등 동일한 내용을 다수 포함





include directive

Confidential

❖ include directive(TODO: 15 - 17)

- 현재 페이지에 다른 JSP 파일의 내용을 삽입하는 형태로 재활용
  - ◆ include directive를 활용한 삽입은 결국 하나의 Servlet 을 생성 → member 및 local 변수의 공용
- include 되는 페이지 구성
  - ◆ 개별적으로 실행될 것이 아니므로 전체적인 HTML은 불필요 → 필요한 태그들만 작성
    - 하지만 JSP 페이지의 기능을 수행하려면 page 지시어는 반드시 필요
  - ◆ include 하는 페이지에 사용될 공통 모듈 포함 : root EL 변수, 공통 CSS, JS 파일 등

```jsp
<!-- /fragments/header.jsp -->
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

<%-- 페이지들이 가져갈 공통적인 내용들 위치 --%>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js" ></script>

<%@ taglib prefix="c" uri="jakarta.tags.core"%>
<c:set var="root" value="${pageContext.request.contextPath }" />
<h1>Welcome to SSAFY</h1>
<hr />
```

Confidential

❖ header.jsp에 다음의 링크를 구성

메뉴	대상 경로
홈으로	/
멤버 목록	/auth?action=member-list
목록 초기화	/member?action=member-list-reset
멤버 가입	/member?action=member-regist-form

include directive

Confidential

❖ include 하는 페이지

- include directive 사용

```
<%@ include file="/target_file"%> <!-- 절대경로 / context root -->
```

- include 된 내용이 통합됨: member는 member로, local 영역은 local로

절대 경로의 기준이 헷갈릴 때 / 기준 판단하기!
- html이 원래 가지고 있던 것들(a, form)은 container root,
- servlet 등에서 새롭게 추가된 것들은 context root!



```
out.write("<!DOCTYPE html>\n");
out.write("<html>\n");
out.write("<head>\n");
out.write("<meta charset=\"UTF-8\">\n");
out.write("<title>Insert title here</title>\n");
out.write("</head>\n");
out.write("<body>\n");
out.write("<div class=\"container\">\n");
out.write("<!-- /fragments/header.jsp -->\n");
out.write("<link href=\"https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css\" rel=\"stylesheet\">\n");
out.write("<script src=\"https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js\" ></script>\n");
out.write("<h1>Welcome to our HRM</h1>\n");
out.write("<hr />");
out.write(" <h2>jsp test </h2>");
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

Member 목록 조회

Confidential

❖ Member 목록 조회

Welcome to SSAFY

홈으로 | 멤버 목록 | 멤버 가입 | 목록 초기화

`/auth?action=member-list`
`/member?action=member-list-reset`

멤버 목록

no	name	email
100	001	099@ssafy.com
99	002	098@ssafy.com
98	003	097@ssafy.com
97	004	096@ssafy.com
96	005	095@ssafy.com

Confidential

❖ FileMember Reset(TODO: 18)

- FileMemberDao

```
public synchronized void reset() {
  int length = 100;
  members.clear();

  for (int i = 0; i < length; i++) {
    members.add(new Member(i, String.format("%03d", length - i), String.format("%03d@ssafy.com", i), "1234", "USER"));
  }
  members.add(new Member(100, "관리자", "admin@ssafy.com", "1234", "ADMIN"));
  System.out.println("멤버 초기화 완료 " + members.size() + " :" + members.get(0));
}
```

- MemberController

```
private void memberListReset(HttpServletRequest request, HttpServletResponse response)
                                                    throws ServletException, IOException {
  mService.reset();
  HttpSession session = request.getSession();
  session.setAttribute("alertMsg", "회원 목록이 초기화되었습니다.");
  redirect(request, response, "/");
}
```

Controller 작성

Confidential

❖ Member List (TODO: 19)

- FileMemberDao

```
@Override
public synchronized List<Member> search() {
  // 완전히 새로운 리스트 생성으로 thread safety 보장, 읽기 전용으로 설정
  return Collections.unmodifiableList( new ArrayList<>(members));
}
```

- AuthController

```
private void memberList(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
  List<Member> members = mService.search();
  request.setAttribute("members", members);
  forward(request, response, "/member/member-list.jsp");
}
```

- member-list.jsp

```
<c:forEach items="${members }" var="item">
  <tr>
    <td>${item.mno }</td><td>${item.name }</td>
    <td><a href="${root }/auth?action=member-detail&email=${item.email}">${item.email }</a></td>
  </tr>
</c:forEach>
```

내일 수업을 위해서..

Confidential

❖ 실습 파일 설치

- 수업 git에서 BE_05_lab.zip 가져오기
- 사용할 workspace에 압축 풀기(여기에 압축 풀기)



- STS : [File] - [Open Projects from File System]
  - ◆ index.jsp 동작 확인

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

내일 방송에서 만나요!

삼성청년SW·AI아카데미